



Class: - B.E.D.S.
Subject: Natural Language Processing

Semester: -VII
A.Y: - 2026-27

Experiment No.1

Tokenization

Aim: To apply various text processing techniques for a given text (Tokenization, filtration and Script Validation).

Software used: Google, Colab – Python

Theory:

Tokenization is the process of breaking down text into individual units called tokens. Tokens can be:

- Words: Tokenization can split text into individual words, such as "The quick brown fox" becoming ["The", "quick", "brown", "fox"].
- Phrases: Tokenization can group words into phrases, like "The quick brown fox" becoming ["The quick brown fox"].
- Characters: Tokenization can split text into individual characters, like "hello" becoming ["h", "e", "l", "l", "o"].
- Sub words: Tokenization can split words into subwords, like "unbreakable" becoming ["un", "break", "able"].

Tokenization is crucial for text analysis, processing, and machine learning tasks, such as:

- Sentiment analysis
- Text classification
- Language translation
- Speech recognition

Script Validation:

Script validation is the process of analyzing and verifying the structure and syntax of code or scripts. It ensures that the code follows the rules and conventions of the programming language. Script validation involves:

- Syntax analysis: checks the code's syntax and structure, like ensuring parentheses are balanced and keywords are used correctly.
- Semantic analysis: checks the code's meaning and functionality, like ensuring variables are declared before use.



Class: - B.E.D.S.

Semester: -VII

Subject: Natural Language Processing

A.Y: - 2026-27

- Lexical analysis: checks the code's tokens and symbols, like ensuring keywords are spelled correctly.

Script validation is essential for:

- Compiler design
- Code analysis and verification
- Debugging
- Code optimization

Regular Expressions (Regex):

Regex is a powerful tool for pattern matching and tokenization. It allows you to define patterns to match text, like finding all occurrences of a specific word or phrase. Regex is used in tokenization, script validation, and text processing.

Context-Free Grammars:

Context-free grammars are a set of rules that define the structure of a programming language. They specify how symbols can be combined to form valid code. Context-free grammars are used in script validation, compiler design, and syntax analysis.

Finite State Automata:

Finite state automata (FSA) are a mathematical model for recognizing patterns in text. They can be used for tokenization, script validation, and text processing. FSA can be thought of as a simple machine that reads input text and changes state based on the input.

Here are some more examples of tokenization:

- Text tokenization: breaking down text into individual words or tokens, such as:
"Hello world, this is a test sentence." → ["Hello", "world", "this", "is", "a", "test", "sentence"]
- Sentence tokenization: breaking down text into individual sentences, such as:
"Hello world, this is a test sentence. Another sentence here." → ["Hello world, this is a test sentence.", "Another sentence here."]
- Subword tokenization: breaking down words into smaller subwords or tokens, such as:
"un-break-able" → ["un", "break", "able"]
- Character tokenization: breaking down text into individual characters such as: **"Hello" → ["H", "e", "l", "l", "o"]**



Class: - B.E.D.S.
Subject: Natural Language Processing

Semester: -VII
A.Y: - 2026-27

1. Tokenization in NLP: tokenization is a crucial step in many NLP tasks, such as language translation, sentiment analysis, and text summarization.
2. Tokenization in programming: tokenization is used in programming languages to break down code into individual tokens, such as keywords, identifiers, and symbols.

How sent_tokenize works?

The sent_tokenize function uses an instance of PunktSentenceTokenizer from the nltk.tokenize.punkt module, which is already been trained and thus very well knows to mark the end and beginning of sentence at what characters and punctuation.

How word tokenize works?

word_tokenize() function is a wrapper function that calls tokenize() on an instance of the TreebankWordTokenizer class.

To run the NLP python program, (NLTK) natural language toolkit has to be installed in your system.

The NLTK module is a massive tool kit, aimed at helping you with the entire Natural Language Processing (NLP) methodology.

In order to install NLTK run the following commands in your terminal. sudo

```
pip install nltk
```

Then, enter the python shell in your terminal by simply typing python.

```
Type import nltk  
nltk.download('all')
```

The above installation will take quite some time due to the massive number of tokenizers, chunkers, other algorithms, and all of the corpora to be downloaded

Some terms that will be frequently used are:

- Corpus – Body of text, singular. Corpora is the plural of this.
- Lexicon – Words and their meanings.
- Token – Each “entity” that is a part of whatever was split up based on rules. For examples, each word is a token when a sentence is “tokenized” into words. Each sentence can also be a token, if you tokenized the sentences out of a paragraph.

Conclusion:

We have understood the importance of text preprocessing. Tokenizing involves splitting sentences and words from the body of the text. One can think of token as parts like a word is a token in a sentence, and a sentence is a token in a paragraph.